

IRENT — FT_TRANSCENDENCE PROJECT PLAN

1. What we're building

irent: a web platform where students can look for a roommate and housing. Students browse rental listings, message owners directly, and cut out brokers. This is our ft_transcendence project — the final Common Core project (5-person team, ~8 weeks).

2. Tech stack (final — no debate needed)

LAYER	TOOL	WHY
Frontend	React	Listed by name in the subject. Biggest community, easiest to find help.
Backend	Express (Node.js)	Listed by name in the subject. Simplest backend framework.
Database	SQLite	A real database stored in one file. No server setup needed.
Real-time	Socket.io	Built on WebSockets (needed for chat and alerts).
Containers	Docker Compose	Required by subject. One command runs everything.

Everyone codes in **JavaScript** only. No second language to learn.

3. Modules we're implementing (14 points total)

MODULE	POINTS	WHAT IT IS IN IRENT
Frontend + backend framework	2	Automatic — using React + Express
Standard user management	2	Signup, login, profile page, and avatar upload
Public API (listings)	2	GET/POST/PUT/DELETE <code>/api/listings</code> with API key + rate limits
User interaction	2	Chat between students & owners, profile views, saved contacts
OAuth login	1	"Login with Google"
File upload	1	Upload, preview, and delete listing photos
Notifications	1	"New message" or "New listing" alerts
Advanced search	1	Filter listings by price, location, etc.
GDPR data controls	1	Users can export or delete their own data
Multiple languages	1	French / Arabic / English

Total: 14 points. Do not add extra modules until this list works perfectly. Keep it simple.

4. Team roles & tasks

Person 1 — Backend & Auth Lead (Tech Lead)

- Set up the code repo, Docker Compose file, and env files.
- Build the Express and SQLite skeleton.
- Database setup: users, listings, messages, favorites.
- Signup and login (secure passwords with bcrypt).
- Profile updates and avatar upload.
- **Learns:** Express, SQLite, password security.

Person 2 — Frontend Lead

- React setup, page routing, and top menu.
- Listing search page and listing detail page.
- Profile page design.
- Search filters (price/location) linked to backend.
- **Learns:** React, React Router, fetching data.

Person 3 — Realtime & Chat

- Chat system: send and receive messages.
- Notification system: alerts for new messages or listings.
- Socket.io setup on server and browser.
- **Learns:** Socket.io and live events.

Person 4 — Listings & Uploads

- Listings logic (create, edit, delete).
- The 5 required public API endpoints (with keys and limits).
- Photo upload system (check size/type, preview, delete).
- **Learns:** Express routes, file uploads (Multer).

Person 5 — PM + Compliance

- Keep the project focused. Decide what to cut if time is short.
- Google login setup.
- Language setup (French/Arabic/English).
- GDPR pages, Privacy Policy, Terms of Service (mandatory).
- Lead meetings, track progress, write the README.
- Ensure everyone understands all parts of the project.
- **Learns:** OAuth login, translation tools.

5. Timeline (8 weeks)

WEEK	FOCUS
1	Person 1 builds the skeleton (React ↔ Express ↔ SQLite in Docker). Everyone else tests it.
2–3	P1: login. P2: listing pages. P4: listing API. P3: chat backend starts.
4–5	P2: search. P3: notifications. P4: photo upload. P5: Google login.
6	P5: languages, GDPR, and legal pages.
7	Integration week — everyone connects their parts and fixes bugs together.
8	README, bug fixes, practice talking about every module (defense prep).

6. Git rules (subject requires this)

- Every team member must have real commits — no one pushing everyone else's code.
- Write clear commit messages (explain what changed, avoid "fix" or "update").
- Use GitHub Issues or a shared board to track who is doing what.

7. Non-negotiables (project will fail without these)

- `docker compose up` must run everything at once.
- Privacy Policy and Terms of Service pages must exist with real text.
- Multiple users must be able to log in and interact at the same time without bugs.
- No errors or warnings in the browser console.
- Use HTTPS for any outside connection to the backend.